Tối ưu dự báo giá dầu bằng mô hình thống kê, học máy và học sâu

Vũ Minh Đức[1], Nguyễn Thị Thúy Vy[1], Phan Thị Tường Vi[1] và Lê Thị Ánh Xuân[1]

Khoa Hệ thống Thông tin, Trường Đại học Công nghệ Thông tin,
Đại học Quốc gia TP.HCM, Việt Nam
22520277@gm.uit.edu.vn, 22521706@gm.uit.edu.vn, 22521658@gm.uit.edu.vn, 22521714@gm.uit.edu.vn

Abstract Dầu mỏ là một trong những tài nguyên quan trọng hàng đầu đối với nền kinh tế toàn cầu. Dự báo giá dầu đóng vai trò then chốt trong việc hỗ trợ ra quyết định đầu tư và hoạch định chính sách. Trong nghiên cứu này, chúng tôi thu thập dữ liệu giao dịch giá dầu từ năm 2000 đến năm 2024 và áp dụng các mô hình dự báo bao gồm Holt-Winters, ARIMA, ARIMAX, Random Forest, XGBoost, CatBoost và LSTM. Mô hình LSTM được triển khai theo hai hướng: đơn biến và đa biến nhằm khai thác thông tin chuỗi thời gian cũng như mối quan hệ giữa các đặc trưng. Hiệu suất của các mô hình được so sánh dựa trên các chỉ số MAE, MSE, RMSE và MAPE. Kết quả cho thấy Random Forest mang lại sai số dự báo thấp nhất so với các mô hình còn lại.

Keywords: Holt-Winters · ARIMA · ARIMAX · CatBoost · Random Forest · XGBoost · LSTM · dự báo giá dầu

1 Giới thiệu

Trong những thập kỷ qua, dầu mỏ đã trở thành một trong những hàng hóa chiến lược quan trọng nhất trong nền kinh tế toàn cầu, ảnh hưởng không chỉ đến thị trường năng lượng mà còn đến các chỉ số kinh tế rộng lớn hơn như lạm phát, chi phí sản xuất và nguồn thu quốc gia. Chính vì thế, việc dự báo giá dầu đã trở thành một nhiệm vụ quan trọng đối với các nhà hoạch định chính sách, nhà đầu tư và các nhà nghiên cứu [1]. Tuy nhiên, việc dự báo giá dầu vẫn luôn là một bài toán thách thức do tính biến động cao của thị trường dầu mỏ và sự ảnh hưởng của nhiều yếu tố kinh tế, địa chính trị và tâm lý thị trường. Như Hamilton [2] đã chỉ ra, giá dầu không chỉ bị tác động bởi các yếu tố cơ bản như cung – cầu mà còn bởi các yếu tố phi cơ bản như bất ổn chính trị hay tâm lý thị trường.

Nghiên cứu này hướng đến việc giải quyết bài toán dự báo giá đóng cửa của dầu thô bằng cách sử dụng kết hợp các kỹ thuật cổ điển và học máy. Các phương pháp được áp dụng và so sánh bao gồm Holt-Winters, ARIMA, ARIMAX, Random Forest, XGBoost, CatBoost và mạng nơ-ron hồi tiếp dài ngắn (LSTM). Mục tiêu của nghiên cứu là xác định mô hình phù hợp nhất để

dự báo giá dầu trong các bối cảnh thị trường khác nhau nhằm hỗ trợ ổn định kinh tế vĩ mô, hỗ trợ doanh nghiệp trong việc lập kế hoạch đầu tư và tối ưu hóa các chiến lược phát triển năng lượng dài hạn.

Tuy nhiên, việc dự báo giá dầu vẫn gặp phải nhiều thách thức do dữ liệu phi tuyến và không dừng, sự xuất hiện của các cú sốc đột ngột, và sự tương tác phức tạp giữa các yếu tố tác động. Yu et al. [3] cũng nhấn mạnh rằng độ tin cậy của các biến ngoại lai trong các mô hình đa biến vẫn là một vấn đề cần được cân nhắc kỹ lưỡng. Để giải quyết vấn đề này, nghiên cứu đề xuất một khung mô hình kết hợp giữa các phương pháp đơn và đa biến. Các mô hình sẽ được huấn luyện và kiểm tra trên giá đóng cửa của năm loại dầu khác nhau và hiệu suất của các mô hình sẽ được đánh giá bằng các chỉ số lỗi tiêu chuẩn như MAE, MSE, RMSE và MAPE. Thông qua việc so sánh này, nghiên cứu kỳ vọng sẽ mang lại cái nhìn thực tiễn về khả năng ứng dụng của từng phương pháp trong điều kiện thị trường năng lượng đầy biến động.

2 Các nghiên cứu liên quan

Dự báo giá dầu khí là bài toán quan trọng trong kinh tế năng lượng, thu hút nhiều nghiên cứu với các phương pháp từ thống kê truyền thống đến học máy hiện đại.

Trong các phương pháp truyền thống, mô hình Holt-Winters và ARIMA được sử dụng rộng rãi nhờ khả năng xử lý các yếu tố xu hướng và mùa vụ trong dữ liệu thời gian. Holt-Winters hiệu quả với dữ liệu có mùa vụ nhưng nhạy cảm với ngoại lệ. Một số cải tiến như dạng nhân [4] hay WMA [5] giúp tăng độ chính xác. ARIMA thích hợp với chuỗi tuyến tính, cơ chế sai phân giúp ổn định chuỗi không dừng; ARIMA(1,1,1) đạt MAPE 2.8% [6], tốt hơn Holt-Winters. SARIMA/SARIMAX cải thiện chính xác nhờ xử lý mùa vụ và biến ngoại sinh [7], [8].

Bên cạnh đó, các mô hình học máy và học sâu như Random Forest, XGBoost, CatBoost và LSTM được khai thác nhờ khả năng xử lý quan hệ phi tuyến và tương tác phức tạp giữa các đặc trưng đầu vào. Random Forest dự báo tốt dữ liệu đa biến và hỗ trợ chọn đặc trưng cho mô hình lai ARIMA-LSTM [9]. XGBoost vượt SVM, ARIMAX về tốc độ và độ chính xác trong dự báo giá dầu [10]; XGBoost-LSTM (kết hợp Attention) cho MAE thấp nhất [11]. CatBoost hỗ trợ chọn đặc trưng, kết hợp BDLSTM nâng độ chính xác [12]. LSTM xử lý tốt chuỗi phi tuyến, phụ thuộc dài hạn; các nghiên cứu kết hợp LSTM với ARMA, DBN hoặc XGBoost giúp nâng cao độ chính xác dự báo giá dầu [13], [14].

Nhìn chung, mô hình truyền thống phù hợp với chuỗi tuyến tính, có yếu tố mùa vụ; trong khi đó, học máy và học sâu mạnh trong xử lý phi tuyến và phụ thuộc dài hạn.

Nghiên cứu này hướng đến cách tiếp cận tổng hợp và hệ thống hơn, đồng thời triển khai và đánh giá hiệu quả của cả mô hình thống kê truyền thống (Holt-Winters, ARIMA, ARIMAX) và các mô hình học máy hiện đại (Random Forest Regression, XGBoost, CatBoost, LSTM) trên cả dạng đơn biến và đa biến. Sự kết hợp này cho phép so sánh trực tiếp hiệu suất giữa các nhóm phương pháp

trong điều kiện thị trường có độ nhiễu cao và nhiều yếu tố tác động, từ đó góp phần định hướng lựa chọn mô hình phù hợp cho các bài toán dự báo giá dầu trong tương lai.

3 Phương pháp

3.1 Holt-Winters

Phương pháp Holt-Winters là một kỹ thuật dự báo chuỗi thời gian thường được sử dụng khi dữ liệu thể hiện cả xu hướng (trend) và tính mùa vụ (seasonality). Phương pháp này được phát triển dựa trên các nghiên cứu của Holt (2004) và Winters, và đã được chuẩn hóa trong các tài liệu như Montgomery et al. (2008) và Kalekar (2004) [15].

Mô hình Holt-Winters gồm ba thành phần: level (mức), trend (xu hướng) và seasonality (mùa vụ), với ba tham số làm trơn tương ứng là α, β, γ trong khoảng từ 0 đến 1 [16].

Additive Holt-Winters (AHW)

Dùng khi biên độ mùa vụ gần như không đổi theo thời gian:

$$\hat{y}_{t+h|t} = \ell_t + hb_t + s_{t+h-m(k+1)} \tag{1}$$

$$\ell_t = \alpha(y_t - s_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1}) \tag{2}$$

$$b_t = \beta^*(\ell_t - \ell_{t-1}) + (1-\beta^*)b_{t-1} \tag{3}$$

$$s_t = \gamma(y_t - \ell_{t-1} - b_{t-1}) + (1-\gamma)s_{t-m} \tag{4}$$

Multiplicative Holt-Winters (MHW)

Dùng khi biên độ mùa vụ tỷ lệ thuận với mức độ (level) của chuỗi:

$$\hat{y}_{t+h|t} = (\ell_t + hb_t) \cdot s_{t+h-m(k+1)} \tag{5}$$

$$\ell_t = \alpha \cdot \frac{y_t}{s_{t-m}} + (1-\alpha)(\ell_{t-1} + b_{t-1}) \tag{6}$$

$$b_t = \beta^* \cdot (\ell_t - \ell_{t-1}) + (1-\beta^*)b_{t-1} \tag{7}$$

$$s_t = \gamma \cdot \frac{y_t}{\ell_{t-1} + b_{t-1}} + (1-\gamma)s_{t-m} \tag{8}$$

Mô hình Holt-Winters được triển khai theo hai biến thể: phương pháp cộng (AHW) và phương pháp nhân (MHW), phù hợp cho chuỗi thời gian có xu hướng và tính mùa vụ rõ rệt. Để cải thiện độ chính xác, phương pháp trung bình trượt có trọng số (WMA) được áp dụng để khởi tạo các giá trị ban đầu thay cho phương pháp truyền thống, dựa trên nguyên tắc ưu tiên thông tin từ các quan sát gần nhất [17].

Quá trình tối ưu hóa tham số được thực hiện thông qua phương pháp tìm kiếm lưới (Grid Search) để xác định bộ tham số tối ưu. Tất cả các tổ hợp tham số

khả thi được sinh ra từ các tập giá trị xác định trước, bao gồm: hệ số làm mượt mức (α), xu hướng (β), mùa vụ (γ), kiểu xu hướng (additive hoặc multiplicative), kiểu mùa vụ (additive hoặc multiplicative), chu kỳ mùa vụ (7, 30, 90, 180, 365), xu hướng suy giảm (có hoặc không), và cửa sổ trung bình trượt (3, 5, 7). Mỗi tổ hợp được đánh giá dựa trên chỉ số lỗi dự báo, từ đó xác định bộ tham số tối ưu nhằm nâng cao độ chính xác của mô hình.

Bảng 1: Thông số cấu hình mô hình Holt-Winters cho từng loại dầu

Loại dầu	**seasonal_periods**	**trend**	**seasonal_type**	**damped**	**wa_window**
Brent_Crude_Oil	7	add	add	false	3
Crude_Oil	365	add	add	false	3
Heating_Oil	90	mul	add	false	3
Natural_Gas	180	add	add	false	7
RBOB_Gasoline	30	add	add	false	5

Bảng 2: Tham số trong mô hình Holt-Winters cho từng loại dầu

Loại dầu	**alpha**	**beta**	**gama**
Brent_Crude_Oil	0.9	0.30	0.1
Crude_Oil	0.9	0.1	0.5
Heating_Oil	0.1	0.1	0.1
Natural_Gas	0.9	0.1	0.1
RBOB_Gasoline	0.1	0.1	0.1

3.2 ARIMA (Autoregressive Integrated Moving Average)

Mô hình ARIMA lần đầu tiên được đề xuất bởi Box và Jenkins trong công trình kinh điển *Time Series Analysis: Forecasting and Control* [19], bao gồm ba thành phần chính: AR (tự hồi quy), I (tích hợp), và MA (trung bình động).

Auto regression: Mô hình tự hồi quy bậc p (viết tắt là AR(p)) là một quá trình trong đó giá trị hiện tại phụ thuộc tuyến tính vào các giá trị trong quá khứ của chính nó. Dạng tổng quát của mô hình AR(p) được biểu diễn như sau:

$$y_t = \phi_0 + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \epsilon_t, \tag{9}$$

trong đó,

- y_t: giá trị tại thời điểm t
- $y_{t-1}, y_{t-2}, \ldots, y_{t-p}$: các giá trị trễ (lags) tại các thời điểm trước đó
- $\phi_0, \phi_1, \ldots, \phi_p$: các hệ số hồi quy, thể hiện mức độ ảnh hưởng của các giá trị quá khứ đến giá trị hiện tại
- p: số bậc trễ trong mô hình AR(p), quyết định số lượng giá trị quá khứ được sử dụng để dự báo
- ϵ_t: sai số ngẫu nhiên tại thời điểm t, phản ánh chênh lệch giữa giá trị thực tế và giá trị dự báo

Thành phần I – Integrated (Tích hợp): đề cập đến quá trình **lấy sai phân (differencing)** nhằm biến đổi chuỗi không dừng thành chuỗi dừng. Sai phân bậc d được biểu diễn như sau:

$$I(d) = \Delta^d(y_t), \tag{10}$$

trong đó,

- d: số lần sai phân cần thiết để chuỗi trở nên dừng
- $\Delta^d(y_t)$: sai phân bậc d của chuỗi y_t

Thành phần MA – Moving Average (Trung bình trượt): Thành phần MA mô tả mối quan hệ tuyến tính giữa giá trị hiện tại của chuỗi và các sai số ngẫu nhiên tại các thời điểm trước đó. Mô hình MA bậc q được viết dưới dạng:

$$y_t = \mu + \epsilon_t + \theta_1\epsilon_{t-1} + \theta_2\epsilon_{t-2} + \cdots + \theta_q\epsilon_{t-q}, \tag{11}$$

trong đó,

- y_t: giá trị tại thời điểm t
- μ: trung bình của chuỗi (nếu có)
- ϵ_t: sai số ngẫu nhiên tại thời điểm t
- $\epsilon_{t-1}, \ldots, \epsilon_{t-q}$: các sai số ngẫu nhiên tại các thời điểm trễ
- $\theta_1, \ldots, \theta_q$: các hệ số MA thể hiện mức độ ảnh hưởng của các sai số quá khứ
- q: số bậc trễ của mô hình MA

Mô hình ARIMA (p, d, q) Mô hình ARIMA kết hợp ba thành phần: AR (Autoregression), I (Integrated – sai phân), và MA (Moving Average), nhằm mô tả chuỗi thời gian không dừng bằng cách lấy sai phân để đảm bảo tính dừng, sau đó áp dụng mô hình ARMA. Công thức tổng quát:

$$\text{ARIMA}(p,d,q): \quad \Delta^d y_t = \mu + \phi_0 + \sum_{i=1}^{p} \phi_i \Delta^d y_{t-i} + \sum_{j=1}^{q} \theta_j \epsilon_{t-j} + \epsilon_t, \tag{12}$$

trong đó,

- $\Delta^d y_t$: chuỗi đã lấy sai phân bậc d
- μ: trung bình của chuỗi
- ϕ_i: hệ số tự hồi quy bậc i
- θ_j: hệ số trung bình trượt bậc j
- ϵ_t: sai số ngẫu nhiên tại thời điểm t

3.3 ARIMAX (AutoRegressive Integrated Moving Average with Exogenous Variables)

Mặc dù mô hình ARIMA rất hiệu quả trong việc dự báo chuỗi thời gian dựa trên dữ liệu lịch sử, nhưng không thể khai thác thông tin từ các yếu tố bên ngoài có thể ảnh hưởng đến chuỗi. Để khắc phục hạn chế này, mô hình ARIMAX được phát triển bằng cách bổ sung thêm các biến ngoại sinh (exogenous variables) X, giúp cải thiện độ chính xác của dự báo bằng việc kết hợp thông tin ngoài chuỗi thời gian [21].

Phương trình tổng quát của mô hình ARIMAX được viết như sau:

$$\begin{aligned}\text{ARIMAX}(p,d,q,X): \quad \Delta^d y_t &= \mu + \phi_0 + \sum_{i=1}^{p} \phi_i \Delta^d y_{t-i} \\ &+ \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \sum_{k=1}^{m}\sum_{r=1}^{s} \beta_{k,r} X_{k,t-r} + \varepsilon_t, \quad (13)\end{aligned}$$

trong đó,

- $\Delta^d y_t$: chuỗi y_t sau khi lấy sai phân bậc d
- μ: trung bình của chuỗi thời gian
- ϕ_i: hệ số tự hồi quy (AR) bậc i, mô tả ảnh hưởng của các giá trị quá khứ $\Delta^d y_{t-i}$
- θ_j: hệ số trung bình trượt (MA) bậc j, mô tả ảnh hưởng của sai số quá khứ ε_{t-j}
- ε_t: sai số ngẫu nhiên tại thời điểm t
- $X_{k,t-r}$: giá trị của biến ngoại sinh thứ k tại thời điểm $t-r$
- $\beta_{k,r}$: hệ số của biến ngoại sinh $X_{k,t-r}$ tại độ trễ r
- m: số lượng biến ngoại sinh
- s: số độ trễ tương ứng của mỗi biến ngoại sinh

Trong quá trình dự báo, chúng tôi sử dụng hàm `auto_arima` từ thư viện `pmdarima` để tự động lựa chọn cấu hình tối ưu cho cả mô hình ARIMA và ARIMAX. Phạm vi tìm kiếm các tham số (p,d,q) được giới hạn ở $p,q \in [0,5]$, $d \leq 2$, theo khuyến nghị từ tài liệu chính thức của thư viện pmdarima [22]. Cấu hình này được lựa chọn để đảm bảo mô hình không quá phức tạp nhưng vẫn mô tả hiệu quả đặc trưng động của chuỗi thời gian. Đặc biệt, quá trình chọn mô hình sử dụng phương pháp *stepwise search* nhằm tối ưu thời gian huấn luyện và tránh việc tìm kiếm toàn bộ không gian tham số.

Bảng 3 thể hiện các bộ tham số tối ưu (p,d,q) được lựa chọn tự động cho từng loại dầu thô khi sử dụng mô hình ARIMA và ARIMAX.

Bảng 3: Thông số mô hình ARIMA và ARIMAX cho từng loại dầu

Loại dầu	**ARIMA (p, d, q)**	**ARIMAX (p, d, q)**
Brent_Crude_Oil	(0, 1, 1)	(0, 1, 1)
Crude_Oil	(0, 1, 1)	(0, 1, 1)
Heating_Oil	(0, 1, 1)	(0, 1, 1)
Natural_Gas	(0, 1, 0)	(0, 1, 0)
RBOB_Gasoline	(0, 1, 0)	(0, 1, 0)

Đối với mô hình ARIMA, chỉ chuỗi giá đóng cửa (giá cần dự báo) được đưa vào. Trong khi đó, mô hình ARIMAX được mở rộng bằng cách bổ sung thêm các biến ngoại sinh. Các biến này được giả định là có ảnh hưởng đến giá cuối cùng, từ đó góp phần cải thiện độ chính xác của mô hình.

3.4 Random Forest Regression

Random Forest Regression là một thuật toán học máy thuộc nhóm ensemble learning, kết hợp nhiều mô hình học máy yếu, cụ thể là cây quyết định, để tạo ra một mô hình chính xác và ổn định hơn [23].

Random Forest Regression tạo ra n cây quyết định độc lập. Mỗi cây được huấn luyện trên một tập con ngẫu nhiên của dữ liệu huấn luyện. Mỗi lần cây cần chia nhánh, thuật toán chọn ngẫu nhiên m đặc trưng từ tổng số M đặc trưng ban đầu. Mỗi cây cho ra một dự đoán, thuật toán sẽ tính giá trị trung bình các dự đoán từ tất cả các cây để cho ra kết quả cuối cùng.

Để cải thiện hiệu suất dự báo và giảm thiểu hiện tượng overfitting cho các mô hình, chúng tôi tiến hành tối ưu hóa siêu tham số với thư viện GridSearchCV của Python. Tập hợp các siêu tham số được lựa chọn để tối ưu bao gồm: các giá trị 50, 100 và 200 đối với số lượng cây trong rừng; các giá trị 5, 10 và 15 đối với độ sâu tối đa của mỗi cây; các giá trị 2 và 5 đối với số lượng mẫu tối thiểu để chia một nút; các giá trị sqrt, log2 và 0.8 đối với số lượng đặc trưng được xem xét tại mỗi điểm chia.

3.5 XGBoost

XGBoost xây dựng một tập hợp các cây quyết định một cách tuần tự. Mỗi cây mới được thêm vào nhằm giảm sai số (residual) của mô hình hiện tại bằng cách dùng kỹ thuật gradient descent trên một hàm mất mát [24].

Đầu tiên, khởi tạo một mô hình đơn giản, ví dụ:

$$\hat{y}_i^{(0)} = \bar{y} \tag{14}$$

Tại bước thứ t, mô hình dự đoán là:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i) \tag{15}$$

trong đó,

Bảng 4: Các bộ tham số tối ưu nhất đối với từng loại dầu

Loại dầu	**Số cây**	**Độ sâu tối đa của cây**	**Số mẫu tối thiểu để chia nút**	**Số đặc trưng được xem xét tại mỗi điểm chia**
Brent_Crude_Oil	50	10	2	0.8
Crude_Oil	50	10	2	0.8
Heating_Oil	200	15	2	0.8
Natural_Gas	200	15	2	0.8
RBOB_Gasoline	100	10	2	0.8

- $\hat{y}_i^{(t)}$: dự đoán tại vòng lặp thứ t
- $\hat{y}_i^{(t-1)}$: dự đoán tại vòng lặp thứ $t-1$
- $f_t(x_i)$: giá trị do cây quyết định thứ t sinh ra cho mẫu x_i

Hàm mất mát thường dùng là Mean Squared Error (MSE):

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t)}) \tag{16}$$

XGBoost sử dụng hàm mất mát Taylor bậc hai:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t) \tag{17}$$

trong đó,

- $g_i = \dfrac{\partial l(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}$: đạo hàm bậc nhất (gradient) của hàm mất mát tại mẫu thứ i
- $h_i = \dfrac{\partial^2 l(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2}$: đạo hàm bậc hai (Hessian) của hàm mất mát tại mẫu thứ i
- $\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$: hàm phạt độ phức tạp của mô hình, trong đó T là số lá của cây, w_j là trọng số tại lá thứ j, γ và λ là các hệ số điều chuẩn (regularization parameters)

XGBoost tìm f_t sao cho hàm mất mát Taylor bậc hai được giảm nhiều nhất.

Để cải thiện hiệu suất của các mô hình XGBoost, chúng tôi tiến hành tối ưu hóa các siêu tham số bằng cách sử dụng thư viện GridSearchCV của Python. Tập hợp các siêu tham số được lựa chọn để tối ưu bao gồm: các giá trị 100 và 200 đối với số lượng cây được xây dựng; các giá trị 3 và 5 đối với độ sâu tối đa của mỗi cây; các giá trị 0.05 và 0.1 đối với tốc độ học; các giá trị 0.8 và 1.0 đối với tỷ lệ mẫu được chọn ngẫu nhiên từ tập huấn luyện.

Bảng 5: Các bộ tham số tối ưu nhất đối với từng loại dầu

Loại dầu	**Số lượng cây**	**Độ sâu tối đa của cây**	**Tốc độ học**	**Tỷ lệ mẫu ngẫu nhiên**
Brent_Crude_Oil	200	3	0.1	0.8
Crude_Oil	200	3	0.1	1.0
Heating_Oil	100	3	0.1	0.8
Natural_Gas	200	3	0.1	0.8
RBOB_Gasoline	200	3	0.1	1.0

3.6 CatBoost

Catboost (tăng cường theo danh mục) dựa trên khái niệm về kỹ thuật tăng cường theo gradient, trong đó cây quyết định được xây dựng tuần tự để giảm thiểu lỗi và cải thiện dự đoán. Quy trình hoạt động bằng cách xây dựng một cây quyết định và đánh giá lượng lỗi trong dự đoán. Khi cây đầu tiên được xây dựng, cây tiếp theo được tạo ra để sửa lỗi do cây trước đó gây ra. Quy trình này tiếp tục lặp đi lặp lại với mỗi cây mới tập trung vào việc cải thiện dự đoán của mô hình bằng cách giảm lỗi trước đó, quy trình này tiếp tục cho đến khi đạt được số lần lặp được xác định trước. Kết quả là một tập hợp các cây quyết định hoạt động cùng nhau để cung cấp dự đoán chính xác [25].

Mô hình hoạt động qua bốn giai đoạn chính: xử lý đặc trưng, xây dựng cây quyết định, huấn luyện và dự đoán. Trong giai đoạn xử lý đặc trưng, CatBoost tự động xử lý các biến phân loại mà không cần mã hóa thủ công, sử dụng kỹ thuật Target Encoding kết hợp với hoán vị ngẫu nhiên (permutation) để tránh rò rỉ thông tin (data leakage). Giai đoạn xây dựng cây áp dụng kỹ thuật Ordered Boosting nhằm đảm bảo mô hình không sử dụng dữ liệu tương lai trong quá trình học, giúp giảm overfitting và tăng khả năng tổng quát hóa (generalization). Trong quá trình huấn luyện, mô hình liên tục tính toán gradient, xây dựng cây mới dựa trên gradient đó và cập nhật mô hình cho đến khi đạt số vòng lặp tối đa hoặc thoả mãn điều kiện dừng sớm (early stopping). Cuối cùng, trong giai đoạn dự đoán, mô hình tổng hợp đầu ra của tất cả các cây đã học để đưa ra kết quả cuối cùng, hỗ trợ tốt cho cả bài toán phân loại và hồi quy. Hai điểm nổi bật giúp CatBoost vượt trội là khả năng xử lý biến phân loại một cách tự động và chính xác, cùng với việc giảm thiểu rò rỉ thông tin và overfitting thông qua cơ chế boosting có thứ tự.
Công thức:

$$F_t(x) = F_{t-1}(x) + \eta \cdot h_t(x), \tag{18}$$

trong đó,

- $F_t(x)$: mô hình sau vòng lặp thứ t
- η: learning rate

– $h_t(x)$: cây quyết định được huấn luyện để giảm lỗi của $F_{t-1}(x)$

Gradient của hàm mất mát:

∘ Với hàm mất mát MSE:

$$L(y_i, F(x_i)) = \frac{1}{2}(y_i - F(x_i))^2 \rightarrow \nabla = y_i - F(x_i) \tag{19}$$

Target Statistics Encoding (với biến phân loại):

$$\text{Encoded}(x_j) = \frac{\sum^i \in P(j) y_i}{|P(j)|} \tag{20}$$

trong đó,

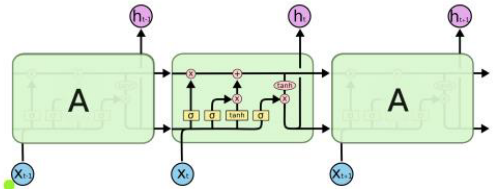
– $P(j)$: tập con các điểm dữ liệu có cùng giá trị phân loại như (x_j) trước (x_j) trong thứ tự hoán vị

Mô hình được hiện thực với các tham số cụ thể như sau: `iterations = 1000`, nghĩa là mô hình sẽ thực hiện tối đa 1000 vòng lặp (tương ứng với 1000 cây quyết định) trong quá trình học; `depth = 6`, cho phép mỗi cây có độ sâu tối đa là 6 nhằm học được các mối quan hệ phức tạp hơn trong dữ liệu; `learning_rate = 0.05`, tức tốc độ học được thiết lập ở mức 0.05 để mô hình cập nhật trọng số chậm hơn, từ đó nâng cao tính ổn định và độ chính xác; cuối cùng, `loss_function = RMSE` được sử dụng làm hàm mất mát, với mục tiêu tối ưu là giảm sai số bình phương trung bình căn bậc hai (Root Mean Squared Error) — một chỉ số phản ánh mức độ chênh lệch giữa giá trị dự đoán và giá trị thực tế.

3.7 LSTM

LSTM một loại mạng RNN đặc biệt thiết kế để ghi nhớ thông tin dài hạn một cách hiệu quả hơn, loại bỏ các thông tin không cần thiết (Hochreiter & Schmidhuber, 1997)[26].

Mục tiêu chính của LSTM: giải quyết hiện tượng "vanishing gradient" mà RNN truyền thống thường gặp và học được các quan hệ dài hạn trong chuỗi [26].

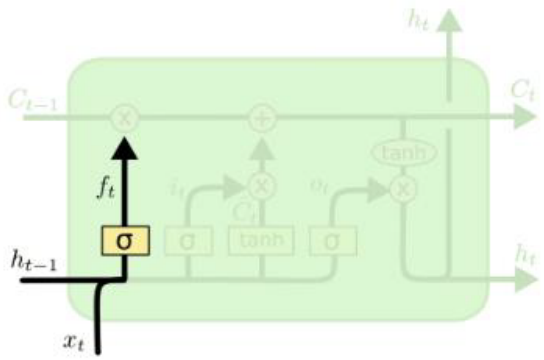


Hình 1: Sơ đồ kiến trúc LSTM

Trong mỗi đơn vị (unit) của mạng LSTM, có ba cổng (gate) sử dụng hàm kích hoạt sigmoid để điều khiển luồng thông tin. Cụ thể, *Forget gate* (f_t) quyết định lượng thông tin cần quên từ trạng thái cell trước đó; *Input gate* (i_t) xác định bao nhiêu thông tin mới sẽ được thêm vào; và *Output gate* (o_t) điều khiển thông tin đầu ra, tức là thông tin sẽ được truyền sang cell tiếp theo. Những cổng này phối hợp với nhau để kiểm soát quá trình học và ghi nhớ thông tin theo thời gian trong chuỗi.

LSTM duy trì hai luồng thông tin chính. Thứ nhất là *Cell state* (C_t), đóng vai trò như bộ nhớ dài hạn, lưu trữ thông tin xuyên suốt chuỗi đầu vào. Trạng thái này được điều chỉnh bởi các giá trị từ f_t và i_t để giữ lại hoặc loại bỏ thông tin phù hợp. Thứ hai là *Hidden state* (h_t), đại diện cho đầu ra tại mỗi bước thời gian, mang thông tin ngắn hạn và được sử dụng để truyền sang bước tiếp theo hoặc đưa vào các tầng tiếp theo như Dense hoặc Softmax để tạo dự đoán.

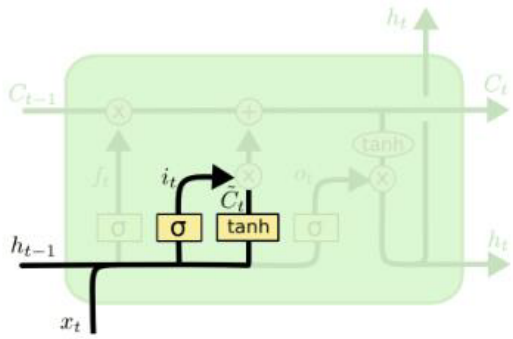
Forget Gate (Cổng quên) f_t trong LSTM là bước đầu tiên, quyết định thông tin nào từ bộ nhớ cũ C_{t-1} cần giữ lại hay loại bỏ, dựa trên đầu vào hiện tại x_t và trạng thái trước h_{t-1}. Nếu output gần 1 → giữ lại, gần 0 → loại bỏ.



Hình 2: Quy trình xử lý đầu vào trong một đơn vị LSTM

$$f_t = \sigma(W_f \cdot [h_{t-1}; x_t] + b_f) \tag{21}$$

Input Gate (Cổng vào) quyết định thông tin mới nào sẽ được thêm vào bộ nhớ. Cổng này gồm hai phần: một tầng sigmoid dùng để xác định phần nào trong thông tin mới nên được cập nhật vào cell state, và một tầng tanh tạo ra các giá trị ứng viên mới, sẵn sàng được thêm vào trạng thái bộ nhớ sau khi được nhân với kết quả từ tầng sigmoid.

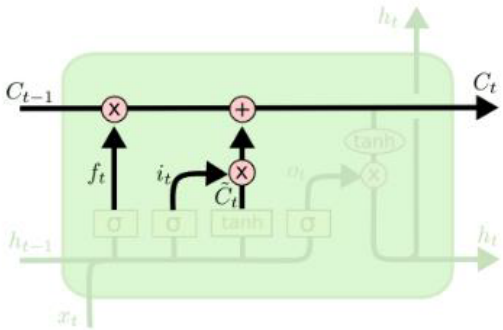


Hình 3: Minh họa hoạt động của cổng vào i_t và quá trình tạo giá trị ứng viên $\tilde{C}_t$ trong một đơn vị LSTM.

$$i_t = \sigma(W_i \cdot [h_{t-1}; x_t] + b_i) \tag{22}$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}; x_t] + b_c) \tag{23}$$

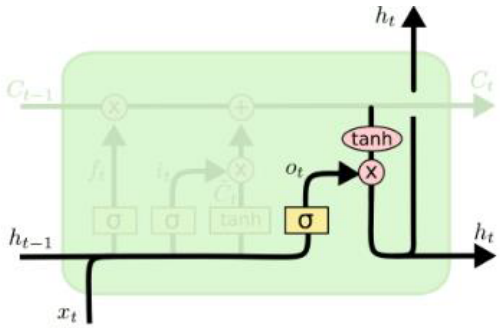
Tại bước xử lý thông tin mới trong LSTM, i_t (input gate) quyết định có nên lưu trữ thông tin mới vào bộ nhớ hay không, trong khi $\tilde{C}_t$ là giá trị ứng viên được tạo ra để thêm vào bộ nhớ, với giá trị nằm trong khoảng từ -1 đến 1 nhờ hàm kích hoạt tanh.
Sau đó, hai phần này được kết hợp để cập nhật trạng thái tế bào C_t. Kết quả là trạng thái tế bào mới C_t phản ánh cả quá khứ được chọn lọc và thông tin mới cần ghi nhớ.



Hình 4: Cập nhật trạng thái bộ nhớ C_t trong đơn vị LSTM thông qua cổng quên f_t, cổng vào i_t và giá trị ứng viên $\tilde{C}_t$.

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \tag{24}$$

Output Gate (Cổng đầu ra) quyết định phần nào của trạng thái bộ nhớ C_t sẽ được đưa ra ngoài tại mỗi bước thời gian. Trước hết, một tầng sigmoid được sử dụng để chọn lọc phần thông tin cần xuất ra. Sau đó, trạng thái C_t được chuẩn hóa về khoảng $[-1, 1]$ bằng hàm tanh. Cuối cùng, hai kết quả này được nhân với nhau để tạo ra đầu ra h_t. Đầu ra này phản ánh phần thông tin đã được chọn lọc kỹ lưỡng từ trạng thái bộ nhớ.



Hình 5: Minh họa hoạt động của cổng đầu ra o_t và quá trình tạo trạng thái ẩn h_t từ trạng thái bộ nhớ C_t trong một đơn vị LSTM.

$$o_t = \sigma(W_{io} \cdot [h_{t-1}; x_t] + b_o) \tag{25}$$

$$h_t = o_t * \tanh(C_t) \tag{26}$$

Tại bước đầu ra của LSTM, o_t quyết định phần nào của trạng thái bộ nhớ sẽ được xuất ra, đóng vai trò như một bộ lọc thông tin. Kết quả cuối cùng là h_t (hidden state mới), được sử dụng để làm đầu vào cho bước thời gian tiếp theo hoặc phục vụ cho mục tiêu dự đoán đầu ra.

Để chuẩn bị dữ liệu đầu vào cho mô hình LSTM, chúng tôi đã xử lý chuỗi thời gian bằng cách áp dụng kỹ thuật cửa sổ trượt (sliding window) với kích thước là 10, nhằm tạo ra các tập dữ liệu đầu vào cho mô hình LSTM. Cụ thể, tập X gồm các chuỗi đầu vào có dạng `(window_size, n_features)`, trong khi tập y là giá trị `close` tại thời điểm kế tiếp. Với mỗi loại dầu, một mô hình LSTM gồm hai tầng được xây dựng, bao gồm một lớp LSTM với 50 đơn vị (units) và một lớp Dense đầu ra có một nút (1 neuron) để dự đoán giá trị `close`. Mô hình được biên dịch với thuật toán tối ưu hóa `adam` và hàm mất mát là `mean_squared_error`. Trong giai đoạn huấn luyện, mô hình được đào tạo trên tập huấn luyện và validation với 100 epochs và batch size bằng 16. Quá trình huấn luyện được ghi lại để phục vụ cho việc đánh giá hiệu quả mô hình sau này.

4 Thực nghiệm và đánh giá

4.1 Bộ dữ liệu

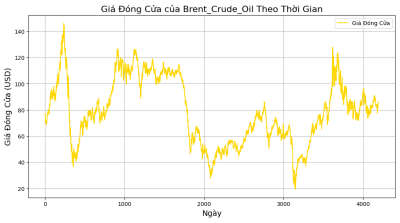
Trong nghiên cứu này, chúng tôi sử dụng bộ dữ liệu **Oil, Gas & Other Fuels Futures Data** - một tập hợp dữ liệu lịch sử về giá giao dịch hàng ngày của các loại nhiên liệu và năng lượng quan trọng, được thu thập từ Yahoo Finance [20]. Cụ thể, dữ liệu bao gồm các hợp đồng tương lai: **Brent Crude Oil (BZ=F)** từ ngày 30/7/2007 đến 24/6/2024, **Crude Oil (CL=F)** từ ngày 23/8/2000 đến 24/6/2024, **Heating Oil (HO=F)** từ ngày 1/9/2000 đến 24/6/2024, **Natural Gas (NG=F)** từ ngày 30/8/2000 đến 24/6/2024, và **RBOB Gasoline (RB=F)** từ ngày 1/11/2000 đến 24/6/2024. Mỗi tập dữ liệu chứa trung bình từ 3.000 đến 6.000 dòng, với các cột thông tin: `Date`, `Open`, `High`, `Low`, `Close`, và `Volume`. Dựa trên bộ dữ liệu này, chúng tôi triển khai hai hướng tiếp cận dự báo: **đơn biến** và **đa biến**. Trong phương pháp đơn biến, chỉ sử dụng giá `Close` để dự đoán chính giá trị `Close` trong tương lai, phù hợp với các mô hình như ARIMA, Holt-Winters và LSTM đơn biến. Ngược lại, phương pháp đa biến sử dụng toàn bộ các đặc trưng `Open`, `High`, `Low`, `Volume` và `Close` làm đầu vào để dự đoán giá `Close`, phù hợp với các mô hình như ARIMAX, XGBoost, Random Forest, CatBoost và LSTM đa biến. Đặc biệt, với các mô hình học máy như XGBoost, Random Forest và CatBoost, chúng tôi còn trích xuất thêm các đặc trưng thời gian từ cột `Date` như `day`, `month`, và `year` nhằm bổ sung thông tin về xu hướng theo mùa vụ và chu kỳ năm.

Chiến lược chia tập dữ liệu được thiết kế phù hợp với từng loại mô hình. Cụ thể, với các mô hình thống kê và học máy như ARIMA, ARIMAX, Holt-Winters, XGBoost, Random Forest và CatBoost, dữ liệu được chia theo tỷ lệ 80% cho huấn luyện (training) và 20% cho kiểm tra (testing). Đối với mô hình LSTM, tỷ lệ chia là 70% cho training, 10% cho validation, và 20% cho testing, kèm theo bước chuẩn hóa dữ liệu đầu vào bằng MinMaxScaler về khoảng [0, 1]. Việc kết hợp các nhóm mô hình truyền thống, học máy và deep learning, cùng với hai chiến lược đơn biến – đa biến, cho phép so sánh hiệu quả dự báo trong nhiều bối cảnh khác nhau. Đồng thời, việc trích xuất đặc trưng thời gian và xử lý dữ liệu đặc thù cho từng mô hình góp phần nâng cao độ chính xác và khả năng tổng quát hóa trong bài toán dự báo giá năng lượng thực tiễn.

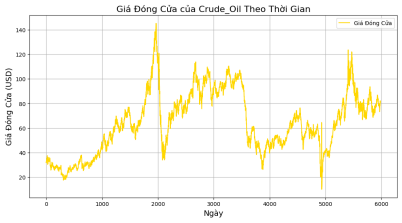
Quá trình xử lý dữ liệu và phân tích dự đoán giá nhiên liệu bao gồm năm bước chính. Đầu tiên, hệ thống đọc tất cả các tệp `.csv` từ thư mục `All_Fuel_Data` và lưu từng bảng dữ liệu (DataFrame) vào một dictionary với tên tệp làm khóa. Tiếp theo, dữ liệu được làm sạch bằng cách kiểm tra và xử lý các giá trị thiếu (NaN), chuyển cột `date` sang kiểu `datetime`, đặt làm chỉ mục thời gian (index), sắp xếp theo thứ tự thời gian và trích xuất thêm thông tin từ ngày như `year`, `month` và `day`. Ở bước xử lý giá trị bất thường, các cột như `open`, `high`, `low`, `close`, và `volume` được kiểm tra để phát hiện các giá trị nhỏ hơn hoặc bằng 0; các giá trị không hợp lệ trong các cột giá sẽ được thay thế bằng trung bình của các giá trị hợp lệ tương ứng. Sau khi xử lý, dữ liệu sạch được lưu lại thành các tệp riêng biệt trong thư mục `processed_data`. Cuối cùng, mô hình dự đoán

được áp dụng để phân tích giá nhiên liệu tương ứng với từng DataFrame, nhằm dự báo xu hướng biến động giá của từng loại nhiên liệu theo thời gian.

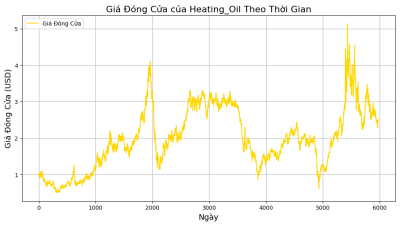
Các biểu đồ sau thể hiện diễn biến giá đóng cửa của các loại dầu thô trong tập dữ liệu, làm cơ sở cho việc đánh giá và so sánh các mô hình dự báo.



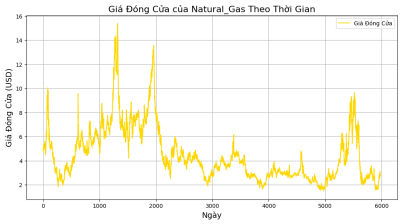
(a) Brent Crude Oil



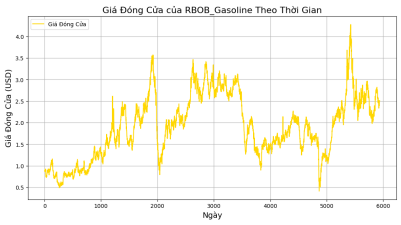
(b) Crude Oil



(c) Heating Oil



(d) Natural Gas



(e) RBOB Gasoline

Hình 6: Giá đóng cửa của các loại dầu theo thời gian

4.2 Các tiêu chí đánh giá

Trong bối cảnh dự báo giá, các chỉ số đánh giá như Mean Squared Error (MSE), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) và Mean Absolute Percentage Error (MAPE) thường được sử dụng để đo lường mức độ sai lệch giữa giá trị thực tế và giá trị dự báo. Cụ thể, MSE và RMSE nhạy cảm với các sai số lớn, trong khi MAE và MAPE cung cấp cái nhìn rõ hơn về mức độ sai lệch trung bình theo đơn vị gốc hoặc phần trăm. Việc kết hợp các chỉ số này giúp đánh giá toàn diện hiệu quả của mô hình dự báo giá [18].

Mean Absolute Error (MAE): Trung bình tuyệt đối sai số giữa giá trị thực và dự đoán, giúp phản ánh mức sai số trung bình theo đơn vị gốc.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{27}$$

Mean Squared Error (MSE): Trung bình bình phương sai số giữa giá trị thực tế và giá trị dự đoán.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{28}$$

Root Mean Squared Error (RMSE): Căn bậc hai của MSE, giúp biểu diễn sai số theo cùng đơn vị với dữ liệu gốc. RMSE cũng nhạy cảm với các sai số lớn.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{29}$$

Mean Absolute Percentage Error (MAPE): Tỷ lệ phần trăm sai số tuyệt đối trung bình, giúp đánh giá mức độ sai số theo phần trăm so với thực tế.

$$\text{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{30}$$

Trong đó, y_i là giá trị thực tế, $\hat{y}_i$ là giá trị dự đoán, và n là số lượng quan sát. Trong nghiên cứu này, mô hình có giá trị MAE, MSE, RMSE và MAPE thấp hơn được xem là có hiệu quả dự báo tốt hơn.

4.3 Kết quả

Bảng 6: Kết quả đánh giá các mô hình dự báo giá dầu

Loại	**Mô hình**	**MAE**	**MSE**	**RMSE**	**MAPE**
Brent Crude Oil	Holt Winters	11.426712	259.431355	16.106873	12.266932
	ARIMA	22.036423	645.993638	25.416405	24.392875
	ARIMAX	2.579950	9.585899	3.096110	2.849378
	CatBoost	1.825866	8.413047	2.900525	1.919944
	Random Forest	**0.779008**	**1.118672**	**1.057673**	**0.894039**
	XGBoost	0.818308	1.226709	1.107569	0.944361
	LSTM (đơn biến)	1.458995	4.577273	2.139456	1.673846
	LSTM (đa biến)	1.454580	4.445830	2.108514	1.666702
Crude Oil	Holt Winters	12.139832	270.630549	16.450853	23.350517
	ARIMA	20.148034	568.476624	23.842748	32.161923
	ARIMAX	4.386737	27.303347	5.225260	6.912405
	CatBoost	1.381456	7.318106	2.705200	3.043908
	Random Forest	**0.715521**	**2.309546**	**1.519719**	**1.422357**
	XGBoost	0.821536	2.507113	1.583386	1.626115
	LSTM (đơn biến)	1.426048	7.757666	2.785259	2.587662
	LSTM (đa biến)	1.454132	8.048632	2.837011	2.671343
Heating Oil	Holt Winters	0.532152	0.494193	0.702988	26.439578
	ARIMA	0.766398	0.911423	0.954685	33.888764
	ARIMAX	0.158409	0.039552	0.198878	7.034731
	CatBoost	0.073587	0.023088	0.151948	2.628864
	Random Forest	**0.030978**	**0.004980**	**0.070568**	**1.130484**
	XGBoost	0.039121	0.008344	0.091345	1.413999
	LSTM (đơn biến)	0.051144	0.006583	0.081135	2.087736
	LSTM (đa biến)	0.049720	0.006579	0.081110	2.014384
Natural Gas	Holt Winters	1.318516	4.151326	2.037480	31.050048
	ARIMA	1.334769	4.576873	2.139363	29.495935
	ARIMAX	0.039552	0.214649	0.463303	6.468048
	CatBoost	0.114553	0.047222	0.217307	2.685821
	Random Forest	**0.054967**	**0.008162**	**0.090342**	**1.503751**
	XGBoost	0.064885	0.009287	0.096371	1.946634
	LSTM (đơn biến)	0.155098	0.056316	0.237310	4.394263
	LSTM (đa biến)	0.133027	0.045422	0.213125	3.500858
RBOB Gasoline	Holt Winters	0.389455	0.298218	0.546094	23.321802
	ARIMA	0.796586	0.901638	0.949546	36.031799
	ARIMAX	0.141084	0.029069	0.170496	6.561645
	CatBoost	0.054250	0.016414	0.128119	2.774617
	Random Forest	**0.031804**	0.006908	0.083113	**1.312024**
	XGBoost	0.036699	0.008933	0.094512	1.511955
	LSTM (đơn biến)	0.042644	**0.003993**	**0.063189**	2.096834
	LSTM (đa biến)	0.043299	0.004030	0.063480	2.115805

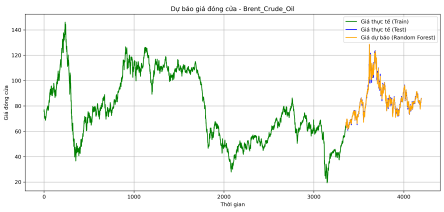
Dựa trên Bảng 6, các mô hình học máy hiện đại như **Random Forest**, **CatBoost**, **XGBoost** và **LSTM** cho hiệu suất vượt trội so với các mô hình truyền thống như **Holt-Winters**, **ARIMA** và **ARIMAX**. Trong đó, **Random Forest** đạt kết quả tốt nhất ở hầu hết các loại dầu (Brent, Crude, Heating, Natural Gas), với sai số MAE, RMSE và MAPE thấp nhất. Đặc biệt, với Brent và Crude Oil, MAE của Random Forest nhỏ vượt trội, cho thấy khả năng khái quát hóa mạnh.

Với RBOB Gasoline, **LSTM đơn biến** đạt RMSE thấp nhất, trong khi **Random Forest** vẫn giữ MAPE thấp hơn, phản ánh độ chính xác tương đối tốt hơn.

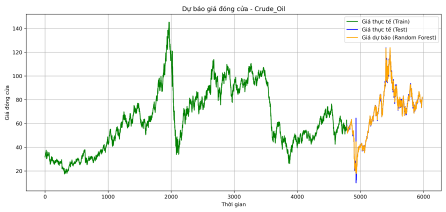
Mô hình truyền thống **ARIMAX** (đa biến) có cải thiện rõ rệt so với **ARIMA** trong tất cả loại dầu.

Ngoài ra, **LSTM đa biến** tốt hơn **đơn biến** ở Brent Crude Oil, Heating Oil và Natural Gas, cho thấy lợi ích của việc khai thác nhiều đặc trưng đầu vào.

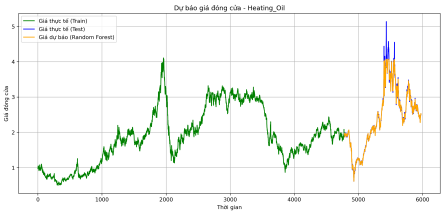
Tổng thể, **Random Forest** là mô hình hiệu quả và ổn định nhất, khẳng định tiềm năng của các phương pháp học máy hiện đại trong dự báo giá dầu.



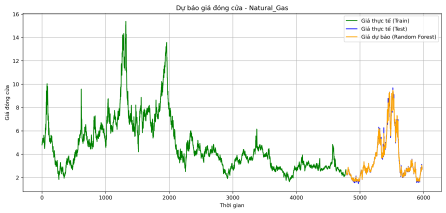
(a) Brent Crude Oil (Random Forest)



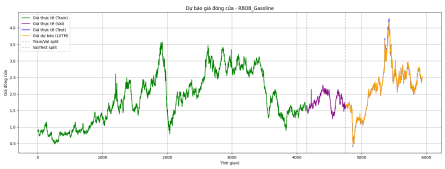
(b) Crude Oil (Random Forest)



(c) Heating Oil (Random Forest)



(d) Natural Gas (Random Forest)



(e) RBOB Gasoline (LSTM đơn biến)

Hình 7: Trực quan dự báo giá đóng cửa của mô hình tốt nhất cho từng loại dầu

5 Kết luận

Trong nghiên cứu này, chúng tôi đã tiến hành so sánh hiệu năng dự báo giá đóng cửa của năm loại dầu (Brent Crude Oil, Crude Oil, Heating Oil, Natural Gas, RBOB Gasoline) bằng các mô hình truyền thống (Holt-Winters, ARIMA, ARIMAX) và các mô hình học máy hiện đại (Random Forest, XGBoost, CatBoost, LSTM đơn biến và đa biến). Kết quả thực nghiệm cho thấy các mô hình học máy, đặc biệt là **Random Forest** có các chỉ số lỗi (MAE, MSE, RMSE, MAPE) thấp nhất trong hầu hết các loại dầu. Mô hình **ARIMAX** thường cho kết quả vượt trội hơn ARIMA đơn biến nhờ tích hợp biến đầu vào bên ngoài, nhưng vẫn kém xa các mô hình học máy về mặt chính xác. Mô hình **LSTM**, đặc biệt ở biến thể **đa biến**, cho thấy cải thiện nhất định so với LSTM đơn biến. Các mô hình truyền thống (đặc biệt là Holt-Winters và ARIMA) gặp khó khăn rõ rệt với chuỗi thời gian phi tuyến hoặc có biến động mạnh.

Nhìn chung, nghiên cứu khẳng định vai trò nổi bật của các phương pháp học máy hiện đại trong bài toán dự báo chuỗi thời gian, đặc biệt khi dữ liệu có tính phi tuyến và chịu ảnh hưởng bởi nhiều yếu tố bên ngoài. Những kết quả này mở ra hướng ứng dụng thực tiễn rộng rãi trong các hệ thống hỗ trợ ra quyết định liên quan đến thị trường năng lượng.

Acknowledgments. Nhóm tác giả xin chân thành cảm ơn ThS. Dương Phi Long, giảng viên Khoa Hệ thống Thông tin, Trường Đại học Công nghệ Thông tin – ĐHQG-HCM, đã tận tình hướng dẫn và hỗ trợ chuyên môn trong suốt quá trình thực hiện nghiên cứu. Nghiên cứu này không nhận được tài trợ tài chính từ bất kỳ tổ chức nào.

Disclosure of Interests. Các tác giả cam kết không có xung đột lợi ích liên quan đến nội dung của bài báo này.

References

1. Zhang, Y., Wang, J., Li, X.: Forecasting crude oil prices with a multi-model approach based on machine learning and statistical models. Energy Economics **89**, 104748 (2020)
2. Hamilton, J.D.: Causes and consequences of the oil shock of 2007–08. Brookings Papers on Economic Activity **2009**(1), 215–261 (2009)
3. Yu, L., Wang, S., Lai, K.K.: Forecasting crude oil price with an EMD-based neural network ensemble learning paradigm. Energy Economics **81**, 233–244 (2019)
4. Heydari, M., Benisi Ghadim, H., Rashidi, M., Noori, M.: Application of Holt-Winters Time Series Models for Predicting Climatic Parameters. Polish Journal of Environmental Studies **29**(1), 617–627 (2019)
5. Croux, C., Gelper, S., Mahieu, K.: Robust exponential smoothing of multivariate time series. Comput. Stat. Data Anal. **54**(12), 2999–3006 (2010)
6. Hansun, S., Charles, V., Indrati, C. R., Subanar: Revisiting the Holt-Winters' Additive Method for Better Forecasting. Int. J. Enterprise Inf. Syst. **15**(2), 43–57 (2019)
7. Udom, P.: A comparison study between time series model and ARIMA model for sales forecasting of distributor in plastic industry. IOSR J. Eng. **4**(2), 32–38 (2014)

8. Mondal, P., Shit, L., Goswami, S.: Study of Effectiveness of Time Series Modeling (ARIMA) in Forecasting Stock Prices. Int. J. Comput. Sci. Eng. Appl. **4**(2), 13–29 (2014)
9. Jain, G., Mallick, B.: A Study of Time Series Models ARIMA and ETS. SSRN Electron. J. (2017)
10. Mohammadi, H., Su, L.: International evidence on crude oil price dynamics: Applications of ARIMA-GARCH models. Energy Econ. **32**(5), 1001–1008 (2010)
11. Box, G. E. P., Jenkins, G. M., Reinsel, G. C.: Time Series Analysis: Forecasting and Control. Wiley (2008)
12. Wang, Y.: Oil Price Forecasting based on Improved SARIMA Model. Highlights Sci. Eng. Technol. **39**, 238–245 (2023)
13. Altincop, H., Oktay, A. B.: Air Pollution Forecasting with Random Forest Time Series Analysis. Proc. 2018 Int. Conf. Data Anal. Process. (2018)
14. Kane, M. J., Price, N., Scotch, M., Rabinowitz, P.: Comparison of ARIMA and Random Forest time series models for prediction of avian influenza H5N1 outbreaks. BMC Bioinformatics **15**(1) (2014)
15. T. Booranawong and A. Booranawong, "Simple and double exponential smoothing methods with designed input data for forecasting a seasonal time series: In an application for lime prices in Thailand," *Int. J. Pure Appl. Math.*, vol. 24, no. 3, pp. 301–310, Nov. 2017.
16. Otexts, *Holt-Winters' Seasonal Method – Forecasting: Principles and Practice*. Available online: `https://otexts.com/fpp2/holt-winters.html`, last accessed 2025/05/25.
17. Hansun, S., Charles, V., Indrati, C. R., Subanar: Revisiting the Holt-Winters' Additive Method for Better Forecasting. Int. J. Enterprise Inf. Syst. **15**(2), 43–57 (2019)
18. Hyndman, R.J., Athanasopoulos, G.: Forecasting: Principles and Practice. 3rd edn. OTexts, Melbourne (2021). `https://otexts.com/fpp3/`
19. Box, G. E. P., Jenkins, G. M., Reinsel, G. C., Ljung, G. M.: *Time Series Analysis: Forecasting and Control*. 5th edn. Wiley, Hoboken (2015)
20. Yahoo Finance: Commodities Market – Oil, Gas Other Futures. `https://finance.yahoo.com/markets/commodities/`, last accessed 2025/06/14.
21. Hyndman, R.J., Athanasopoulos, G.: Forecasting: Principles and Practice. 2nd edn. OTexts, Melbourne (2018). `https://otexts.com/fpp2/`
22. AlkalineML: *pmdarima Documentation*. Available at: `https://alkaline-ml.com/pmdarima/`, last accessed 2025/06/13.
23. GeeksforGeeks: Random Forest Regression in Python. `https://www.geeksforgeeks.org/machine-learning/random-forest-regression-in-python/`, last accessed 2025/06/14.
24. Analytics Vidhya: An End-to-End Guide to Understand the Math behind XGBoost. `https://www.analyticsvidhya.com/blog/2018/09/an-end-to-end-guide-to-understand-the-math-behind-xgb` last accessed 2025/06/14.
25. CatBoost in Machine Learning. `https://www.geeksforgeeks.org/machine-learning/catboost-ml/`, last accessed 2025/06/16.
26. Understanding LSTM Networks. `https://colah.github.io/posts/2015-08-Understanding-LSTMs/`, last accessed 2025/06/17.